Backpropagation in a Single Neuron - Episode 1

Milan Stanarevic

April 22, 2026

Backpropagatin - Prerequisites

We will need few math elements / concepts (High School Level)

- Simple and Complex function Derivative
- Substitution / Chain Rule Derivative
- And two other thing I will not mention here (comes later)

Backpropagation - Refresher - Simple and Complex function Derivative

We will not delve in the definition of Derivative, that does hold our interest and does not bring us closer to our goal.

- Simple function derivative

$$f(x) = x^3 \Rightarrow \frac{d(x^3)}{dx} = 3x^2 \tag{1}$$

$$f(x) = 5x + N \Rightarrow \frac{d(5x + N)}{dx} = 5 \tag{2}$$

Backpropagation - Refresher - Simple and Complex function Derivative

- Complex function derivative and Chain Rule

$$f(x) = (ln(x) + sin(x)^3 \tag{3}$$

$$u = ln(x) + sin(x) \tag{4}$$

$$f(u) = u^3 \tag{5}$$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(u)}{\partial u}\frac{\partial u}{\partial x} \tag{6}$$

$$\frac{\partial f(x)}{\partial x} = 3u^2 * (\frac{1}{x} - cos(x)) \tag{7}$$

$$\frac{\partial f(x)}{\partial x} = 3(ln(x) + sin(x))^2 * (\frac{1}{x} - cos(x)) \tag{8}$$

Backpropagation - Standard Neuron Model

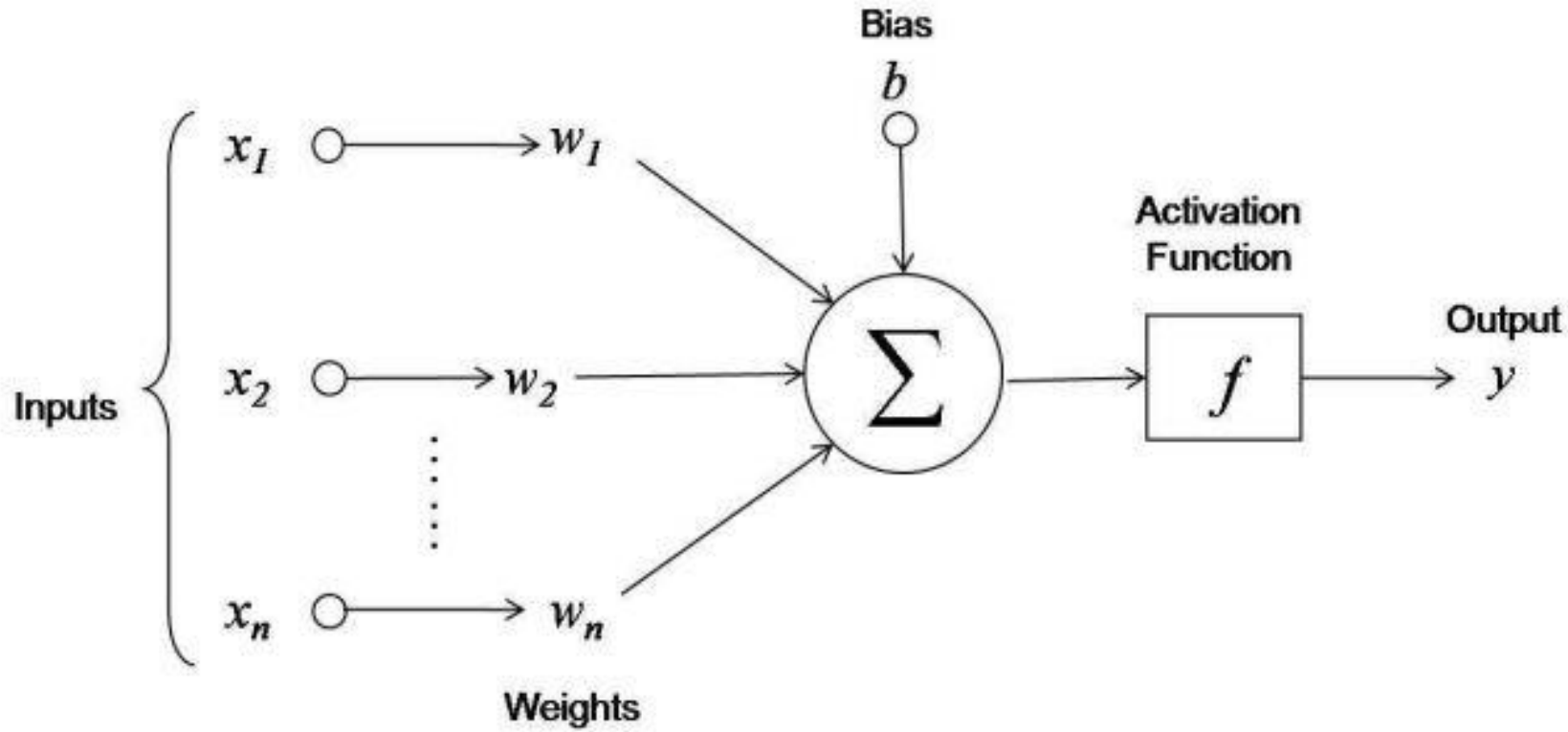

Figure: Standard Neuron Model

Backpropagation - Weighted Input and Activation Function

$$z : \mathbb{R}^n \to \mathbb{R} \tag{9}$$

$$z = \sum_{i=1}^{n} x_i w_i \tag{10}$$

$$a : \mathbb{R} \to \mathbb{R} \tag{11}$$

$$a = \frac{1}{1 + e^{-z}} \tag{12}$$

Backpropagation - Analytical Model & Problem

$$a(z(\overline{x})) \Rightarrow z \circ a \tag{13}$$

$$a = \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + x_3 w_3)}} \tag{14}$$

- Backpropagation is supervised algorithm.
- How to find w_i?

Backpropagation - Model & Problem

Support structure:

- Measure of error / difference in output and target value.

$$C = \frac{1}{2}\sum_{i=1}^{n}(T_i - a_i)^2 \tag{15}$$

- Using gradient descent:

$$w_{new} \leftarrow w_{old} - \eta \nabla C \tag{16}$$

- Where η is the learning rate
- Where ∇ is a vector differential operator

Backpropagation - Putting it all together

- Cost / Loss function

$$C = \frac{1}{2}(T - \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + x_3 w_3)}})^2 \tag{17}$$

$$C = \frac{1}{2}(T - \frac{1}{1 + e^{-z}})^2 \tag{18}$$

$$C = \frac{1}{2}(T - a)^2 \tag{19}$$

- Nector of differential operators

$$\nabla = \begin{bmatrix} \frac{\partial}{\partial x_1} \\ \frac{\partial}{\partial x_2} \\ \frac{\partial}{\partial x_3} \end{bmatrix} \nabla C = \begin{bmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \frac{\partial C}{\partial w_3} \end{bmatrix}$$

Backpropagation - Putting it all together

- Partial derivatives

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_1} \tag{20}$$

$$\frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_2} \tag{21}$$

$$\frac{\partial C}{\partial w_3} = \frac{\partial C}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_3} \tag{22}$$

- Common term

$$\delta = \frac{\partial C}{\partial a}\frac{\partial a}{\partial z} \tag{23}$$

$$\frac{\partial C}{\partial w_i} = \delta\frac{\partial z}{\partial w_i} \tag{24}$$

Backpropagation - Putting it all together

- Partial derivatives

$$\frac{\partial C}{\partial a} = (a - T) \tag{25}$$

$$\frac{\partial a}{\partial z} = a(1 - a) \tag{26}$$

$$\frac{\partial z}{\partial w_i} = x_i \tag{27}$$

- Common term

$$\delta = \frac{\partial C}{\partial a}\frac{\partial a}{\partial z} \Rightarrow (a - T)a(1 - a) \tag{28}$$

$$\frac{\partial C}{\partial w_i} = \delta\frac{\partial z}{\partial w_i} \Rightarrow \delta x_i \tag{29}$$

Backpropagation - Putting it all together

- Vector of differential operators

$$\nabla C = \delta \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

- Gradient descent

$$w_{new} \leftarrow w_{old} - \eta\delta \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \tag{30}$$

Backpropagation - Putting it all together - Pseudocode

Backpropagation in a Multi layer Neuron Network with focus on depth - Episode 2

Milan Stanarevic

April 22, 2026

Backpropagation in a Multi layer Neuron Network: Agenda

- We already know how to backpropagate through a single neuron.
- We will learn how to backpropagate through a multi layer neuron network (no width, just depth).
- We already introduced all the necessary concepts and math elements in the previous episode.
  - Partial derivatives
  - Chain Rule
  - Gradient descent (SGD)
  - Cost / Loss function (MSE)
  - Vector of differential operators (nabla, η)

Backpropagation in a Multi layer Neuron Network: Agenda

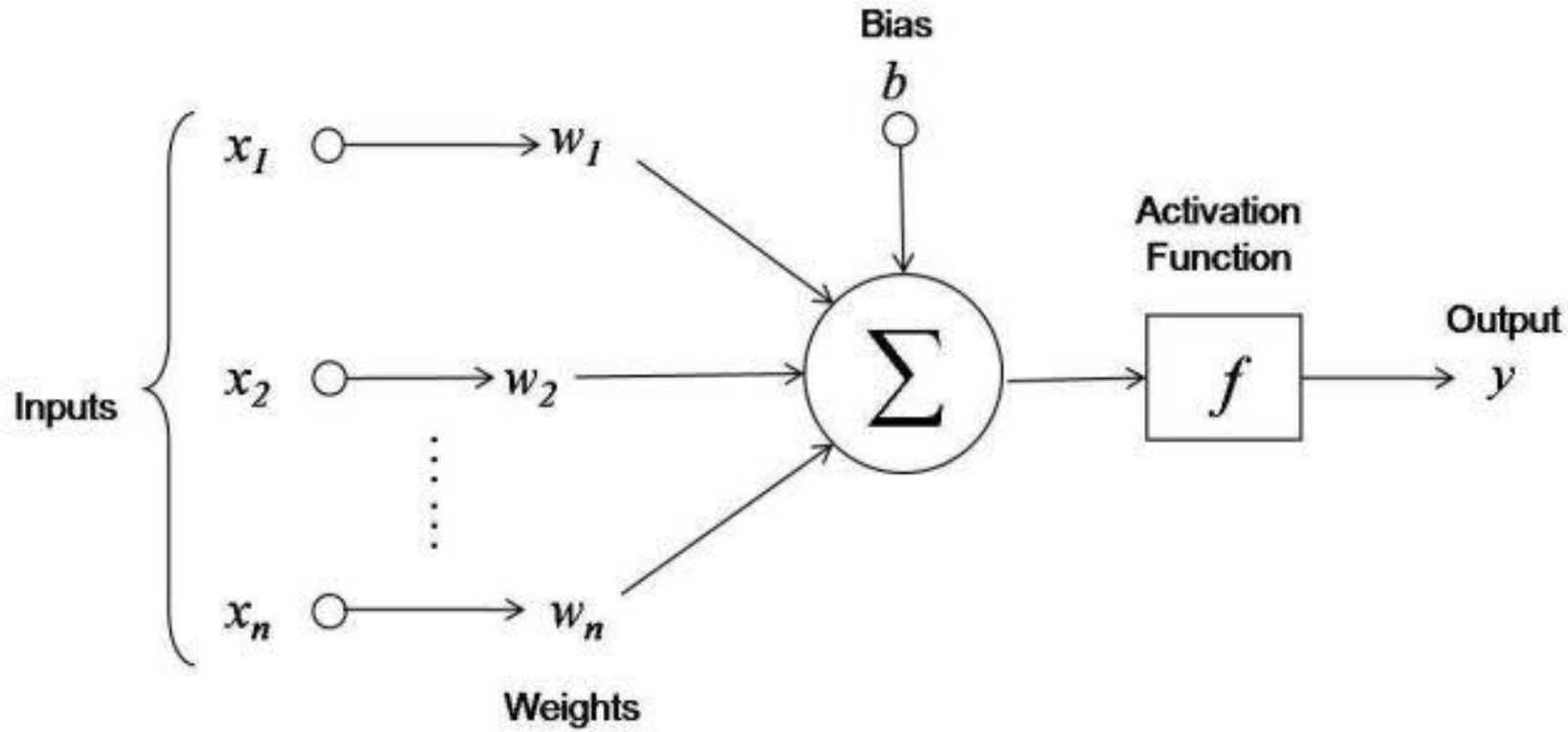

Figure: Standard Neuron Model

Backpropagation in a Multi layer Neuron Network: Strategy

- Apply chain rule to find derivatives for each weight in the network, use MSE as a Loss function (no biases).
- Apply gradient descent to update the weights of the network.
- Feed forward to get the output of the network.
- Check if the output is close to the target value.
- If not,go back to step 1.

Backpropagation in a Multi layer Neuron Network: Gradient Descent

- Derivative of the Cost function with respect to the weights

$$\frac{\partial C}{\partial w_2} \tag{31}$$

$$\frac{\partial C}{\partial w_1} \tag{32}$$

$$\frac{\partial C}{\partial w_{00}}, \frac{\partial C}{\partial w_{01}}, \frac{\partial C}{\partial w_{02}} \tag{33}$$

- Gradient descent

$$w_{new} \leftarrow w_{old} - \eta \nabla C \tag{34}$$

Backpropagation in a Multi layer Neuron Network: Gradient Descent

- Gradient descent

$$\frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial w_2} \tag{35}$$

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1} \tag{36}$$

$$\frac{\partial C}{\partial w_{00}} = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial a_0}\frac{\partial a_0}{\partial z_0}\frac{\partial z_0}{\partial w_{00}} \tag{37}$$

$$\frac{\partial C}{\partial w_{01}} = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial a_0}\frac{\partial a_0}{\partial z_0}\frac{\partial z_0}{\partial w_{01}} \tag{38}$$

$$\frac{\partial C}{\partial w_{02}} = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial a_0}\frac{\partial a_0}{\partial z_0}\frac{\partial z_0}{\partial w_{02}} \tag{39}$$

Backpropagation in a Multi layer Neuron Network: Gradient Descent

- Substitution with layer deltas

$$\delta_2 = \frac{\partial C}{\partial a_2}\frac{\partial a_2}{\partial z_2} \tag{40}$$

$$\delta_1 = \delta_2\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1} \tag{41}$$

$$\delta_0 = \delta_1\frac{\partial z_1}{\partial a_0}\frac{\partial a_0}{\partial z_0} \tag{42}$$

- Compact derivatives per layer

$$\frac{\partial C}{\partial w_2} = \delta_2\frac{\partial z_2}{\partial w_2} \tag{43}$$

$$\frac{\partial C}{\partial w_1} = \delta_1\frac{\partial z_1}{\partial w_1} \tag{44}$$

$$\frac{\partial C}{\partial w_{00}} = \delta_0\frac{\partial z_0}{\partial w_{00}},\ \frac{\partial C}{\partial w_{01}} = \delta_0\frac{\partial z_0}{\partial w_{01}},\ \frac{\partial C}{\partial w_{02}} = \delta_0\frac{\partial z_0}{\partial w_{02}} \tag{45}$$

Backpropagation in a Multi layer Neuron Network: Gradient Descent

- Replace each derivative with its equivalent form

$$\delta_2 = (a_2 - T)a_2(1 - a_2) \tag{46}$$
$$\delta_1 = \delta_2 w_2 a_1(1 - a_1) \tag{47}$$
$$\delta_0 = \delta_1 w_1 a_0(1 - a_0) \tag{48}$$

- Final gradients

$$\frac{\partial C}{\partial w_2} = \delta_2 a_1 \tag{49}$$
$$\frac{\partial C}{\partial w_1} = \delta_1 a_0 \tag{50}$$
$$\frac{\partial C}{\partial w_{00}} = \delta_0 w_{00},\ \frac{\partial C}{\partial w_{01}} = \delta_0 w_{01},\ \frac{\partial C}{\partial w_{02}} = \delta_0 w_{02} \tag{51}$$

Backpropagation in a Multi layer Neuron Network: Gradient Descent

- Gradient descent update rule

$$w_{new} \leftarrow w_{old} - \eta \nabla C \tag{52}$$

- Apply to each weight

$$w_2 \leftarrow w_2 - \eta \delta_2 a_1 \tag{53}$$

$$w_1 \leftarrow w_1 - \eta \delta_1 a_0 \tag{54}$$

$$w_{00} \leftarrow w_{00} - \eta \delta_0 i_0,\ w_{01} \leftarrow w_{01} - \eta \delta_0 i_1,\ w_{02} \leftarrow w_{02} - \eta \delta_0 i_2 \tag{55}$$

Backpropagation in a Multi layer Neuron Network: Pseudocode

```
error = infinity
(i0, i1, i2, T) = sample()
while error > epsilon:
  # run model once and keep intermediates for backprop
  (a0, a1, a2) = forward(i0, i1, i2, w00, w01, w02, w1, w2)

  # backpropagation (training focus)
  delta2 = (a2 - T) * a2 * (1 - a2)
  delta1 = delta2 * w2 * a1 * (1 - a1)
  delta0 = delta1 * w1 * a0 * (1 - a0)

  # gradient descent weight updates
  w2  = w2  - eta * delta2 * a1
  w1  = w1  - eta * delta1 * a0
  w00 = w00 - eta * delta0 * i0
  w01 = w01 - eta * delta0 * i1
  w02 = w02 - eta * delta0 * i2
```